

数据融合与智能分析实验报告 7-8

班级	物联网 2202	学号	223428010210
专业	物联网工程	姓名	陈梓欣
实验成绩			
指导教师	郭振洲	2025 年 6 月 19 日	

一、实验题目：信用卡欺诈行为检测与 SVM 参数条件及其应用

二、实验地点：307-6 雨课堂

三、实验目的：

- 1 理解 Logistic 回归模型应用过程和超参数 C 的意义；
- 2 理解 creditcard.csv 中各项数据含义并进行数据预处理；
- 3 理解并掌握 SVM 原理和参数 C 与 gamma 的具体意义；
- 4 理解并掌握核函数原理和应用；

四、实验内容（完成题目代码及运行结果，代码注释中补充个人信息，以示区别）

1. 对文本文件 creditcard.csv 读取，并对数据进行预览，说明原始数据中存在问题；

```
# 223428010210 陈梓欣
# 读取数据
data = pd.read_csv("creditcard.csv")
data.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267

2. 对原始数据进行预处理，标准化数据，删除不相关的列；

```
# 223428010210 陈梓欣
# 数据预处理
# Amount数据范围大，进行数据标准化，使得每个特征差异相近
# 数据标准化
from sklearn.preprocessing import StandardScaler

data['normAmount'] = StandardScaler().fit_transform(data['Amount'].values.reshape(-1, 1))
data = data.drop(['Time', 'Amount'], axis=1)
data.head()
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V21	V22	V23	V24
0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	0.090794	...	-0.018307	0.277838	-0.110474	0.066928
1	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	-0.166974	...	-0.225775	-0.638672	0.101288	-0.339846
2	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	0.207643	...	0.247998	0.771679	0.909412	-0.689281

```

3 -0.966272 -0.185226 1.792993 -0.863291 -0.010309 1.247203 0.237609 0.377436 -1.387024 -0.054952 ... -0.108300 0.005274 -0.190321 -1.175575
4 -1.158233 0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941 -0.270533 0.817739 0.753074 ... -0.009431 0.798278 -0.137458 0.141267
5 rows × 30 columns

```

3. 对上述数据进行如下操作：

- (1) 分割数据为特征数据与标签数据
- (2) 构建下采样数据
- (3) 在下采样数据集上，分割训练集和测试集；

```

# 223428010210 陈梓欣
# 分割数据，下采样
X = data.loc[:, data.columns != 'Class']
y = data.loc[:, data.columns == 'Class']

# 信用异常数据的数量 number_records_fraud 和对应的样本索引fraud_indices
number_records_fraud = len(data[data.Class == 1])
fraud_indices = np.array(data[data.Class == 1].index)

# 信用正常数据索引
normal_indices = data[data.Class == 0].index

# normal_indices 中进行随机采样并转换成数组
random_normal_indices = np.random.choice(normal_indices, number_records_fraud, replace = False)

# 两个序列合并成
under_sample_indices = np.concatenate([fraud_indices, random_normal_indices])

# 根据序列获取数据
under_sample_data = data.iloc[under_sample_indices, :]

X_undersample = under_sample_data.loc[:, under_sample_data.columns != 'Class']
y_undersample = under_sample_data.loc[:, under_sample_data.columns == 'Class']

# Showing ratio
print("正常数据占比: ", len(under_sample_data[under_sample_data.Class == 0])/len(under_sample_data))
print("异常数据占比: ", len(under_sample_data[under_sample_data.Class == 1])/len(under_sample_data))
print("样本总量为: ", len(under_sample_data))

```

正常数据占比: 0.5
 异常数据占比: 0.5
 样本总量为: 984

```

# 223428010210 陈梓欣
from sklearn.model_selection import train_test_split

# 原始数据划分
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y # 保持原始标签分布一致
)

# 打印分布检查
print("训练集总量:", len(X_train))
print("训练集正负样本比例: ")
print(y_train.value_counts(normalize=True))
print("\n测试集总量:", len(X_test))
print("测试集正负样本比例: ")
print(y_test.value_counts(normalize=True))
print("样本总量:", len(data))

# 划分下采样数据为训练集和测试集 (70/30)，保持类别比例一致
X_train_undersample, X_test_undersample, y_train_undersample, y_test_undersample = train_test_split(
    X_undersample,
    y_undersample,
    test_size=0.3,
    random_state=42,
    stratify=y_undersample
)

print("训练集样本数: ", len(X_train_undersample))
print("测试集样本数: ", len(X_test_undersample))
print("样本总量: ", len(under_sample_data))

```

训练集总量: 199364
 训练集正负样本比例:
 Class
 0 0.998275
 1 0.001725
 Name: proportion, dtype: float64
 测试集总量: 85443

测试集正负样本比例：

```
class
0      0.998268
1      0.001732
Name: proportion, dtype: float64
样本总量: 284807
训练集样本数: 688
测试集样本数: 296
样本总量: 984
```

超参数 C 含义：

4. 说明超参数 C 含义，并定义函数能够根据不同的 C 值进行 Logistic 回归，并返回最佳参数：

```
# 223428010210 陈梓欣
c_param_range = [0.001,0.01,0.1,1,10,100]
score = []
for c in c_param_range:
    print('-----')
    print('C 参数: ', c)
    print('-----')
    print('')
    # 超参数C和L2范数惩罚项 初始化模型
    lr = LogisticRegression(C = c, penalty = 'l2', solver='liblinear')
    # 装载数据，获得模型参数和模型评价指标
    scores = cross_val_score(lr, X_train_undersample, y_train_undersample.values.ravel(), cv=5)
    score.append(scores)

    print(f"各次迭代的决定系数: {scores}")
    print(f"平均决定系数: {scores.mean()}")
```

C 参数: 0.001

各次迭代的决定系数: [0.93 0.89 0.9 0.91 0.88]
平均决定系数: 0.8996826404316091

C 参数: 0.01

各次迭代的决定系数: [0.95 0.92 0.93 0.91 0.92]
平均决定系数: 0.92729228816249

C 参数: 0.1

各次迭代的决定系数: [0.96 0.91 0.94 0.91 0.93]
平均决定系数: 0.9302020522595422

C 参数: 1

各次迭代的决定系数: [0.96 0.92 0.94 0.91 0.93]
平均决定系数: 0.9345604569977786

C 参数: 10

各次迭代的决定系数: [0.96 0.95 0.94 0.91 0.93]
平均决定系数: 0.9374484290701364

C 参数: 100

各次迭代的决定系数: [0.95 0.95 0.94 0.91 0.93]
平均决定系数: 0.9345392996932192

5. 实现把混淆矩阵绘制成热力图功能：

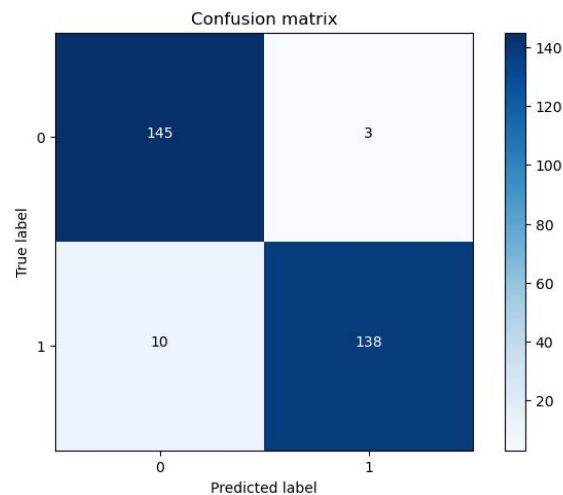
```
# 223428010210 陈梓欣
# 对下采样数据绘制图像
import itertools
lr = LogisticRegression(C = 100, penalty = 'l2')
# lr.fit(X_train_undersample,y_train_undersample.values.ravel())
lr.fit(X_undersample,y_undersample.values.ravel())
#用测试集测试模型 得到测试结果
y_pred_undersample = lr.predict(X_test_undersample)

# Compute confusion matrix
#计算获得混淆矩阵 输入 (真实值, 预测值)
cnf_matrix = confusion_matrix(y_test_undersample,y_pred_undersample)
np.set_printoptions(precision=2)

print("Recall metric in the testing dataset: ", cnf_matrix[1,1]/(cnf_matrix[1,0]+cnf_matrix[1,1]))

# Plot non-normalized confusion matrix
class_names = [0,1]
plt.figure()
plot_confusion_matrix(cnf_matrix
                      , classes=class_names
                      , title='Confusion matrix')
plt.show()
```

Recall metric in the testing dataset: 0.9324324324324325



6. (1) 通过下采样数据集实现 Logistic 回归，并分析结果：

(2) 把下采样数据集的回归模型应用到原始数据中，分析结果：

```
# 223428010210 陈祥欣
# 不做数据平衡处理，训练参数
c_param_range = [0.001,0.01,0.1,1,10,100]
score = []
for c in c_param_range:
    print('-----')
    print('C 参数: ', c)
    lr = LogisticRegression(C = c, penalty = 'l2', solver='liblinear')
    scores = cross_val_score(lr, X_train, y_train.values.ravel(), cv=5)
    score.append(scores)
    print(f"各次迭代的决定系数: {scores}")
    print(f"平均决定系数: {scores.mean()}")
```

```
C 参数: 0.001
各次迭代的决定系数: [1. 1. 1. 1. 1.]
平均决定系数: 0.999122208245932
```

```
C 参数: 0.01
各次迭代的决定系数: [1. 1. 1. 1. 1.]
平均决定系数: 0.9991974473809885
```

```
C 参数: 0.1
各次迭代的决定系数: [1. 1. 1. 1. 1.]
平均决定系数: 0.9992074789805143
```

```
C 参数: 1
各次迭代的决定系数: [1. 1. 1. 1. 1.]
平均决定系数: 0.9992225270088069
```

```
C 参数: 10
各次迭代的决定系数: [1. 1. 1. 1. 1.]
平均决定系数: 0.9992275429343704
```

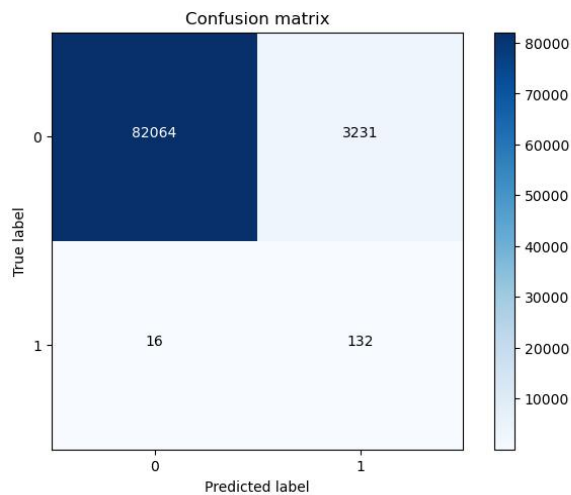
```
C 参数: 100
各次迭代的决定系数: [1. 1. 1. 1. 1.]
平均决定系数: 0.9992275429343704
```

```
# 223428010210 陈祥欣
# 原始数据集训练、测试、结果绘制
lr = LogisticRegression(C = 10, penalty = 'l2')
lr.fit(X_train_undersample, y_train_undersample.values.ravel())
y_pred = lr.predict(X_test.values)

# Compute confusion matrix
cnf_matrix = confusion_matrix(y_test, y_pred)
np.set_printoptions(precision=2)
print("Recall metric in the testing dataset: ", cnf_matrix[1,1]/(cnf_matrix[1,0]+cnf_matrix[1,1]))

# Plot non-normalized confusion matrix
class_names = [0,1]
plt.figure()
plot_confusion_matrix(cnf_matrix, classes=class_names, title='Confusion matrix')
plt.show()
```

Recall metric in the testing dataset: 0.8918918918918919



7.

(1) 建立过采样数据集:

(2) 在过采样数据集上进行 Logistic 回归, 并分析结果:

```
# 223428010210 陈梓欣
# 过采样
import pandas as pd
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
```

```
# 223428010210 陈梓欣
# 读取数据, 分割特征数据集features和标签数据Label
credit_cards=pd.read_csv('creditcard.csv')

columns=credit_cards.columns
# The labels are in the last column ('Class'). Simply remove it to obtain features columns
features_columns=columns.delete(len(columns)-1)

features=credit_cards[features_columns]
labels=credit_cards['Class']
```

```
# 分割成训练集和测试集
features_train, features_test, labels_train, labels_test = train_test_split(features,
                                                                              labels,
                                                                              test_size=0.2,
                                                                              random_state=0)
```

```
# SMOTE过采样
oversampler=SMOTE(random_state=0)
os_features,os_labels=oversampler.fit_resample(features_train,labels_train)
```

```
len(os_labels[os_labels==1])
```

227454

```
# 223428010210 陈梓欣
# 过采样数据集进行训练

# 特征标准化
scaler = StandardScaler()
os_features = scaler.fit_transform(os_features)

# 标签展开为一维数组
os_labels = os_labels.ravel()

# 模型参数测试
c_param_range = [0.001, 0.01, 0.1, 1, 10, 100]
score = []

for c in c_param_range:
    print('-----')
    print('C 参数: ', c)

    lr = LogisticRegression(C=c, penalty='l2', max_iter=1000)
    scores = cross_val_score(lr, os_features, os_labels, cv=5)

    score.append(scores)
    print(f"各次迭代的决定系数: {scores}")
    print(f"平均决定系数: {scores.mean()}")
    print(f"平均决定系数: {scores.mean():.4f}")
```

```

C 参数: 0.001
各次迭代的决定系数: [0.97 0.97 0.97 0.97 0.97]
平均决定系数: 0.9669779384147068
平均决定系数: 0.9670

C 参数: 0.01
各次迭代的决定系数: [0.98 0.97 0.97 0.97 0.97]
平均决定系数: 0.9742585315154202
平均决定系数: 0.9743

C 参数: 0.1
各次迭代的决定系数: [0.98 0.98 0.98 0.98 0.98]
平均决定系数: 0.978342874225849
平均决定系数: 0.9783

C 参数: 1
各次迭代的决定系数: [0.98 0.98 0.98 0.98 0.98]
平均决定系数: 0.9797233735700297
平均决定系数: 0.9797

C 参数: 10
各次迭代的决定系数: [0.98 0.98 0.98 0.98 0.98]
平均决定系数: 0.979921215384512
平均决定系数: 0.9799

C 参数: 100
各次迭代的决定系数: [0.98 0.98 0.98 0.98 0.98]
平均决定系数: 0.9801212557984307
平均决定系数: 0.9801

```

```

# 223428010210 陈梓欣
# 对过采样模型的混淆矩阵绘制
lr = LogisticRegression(C = 10, penalty = 'l2')
lr.fit(os_features,os_labels.ravel())
y_pred = lr.predict(features_test.values)

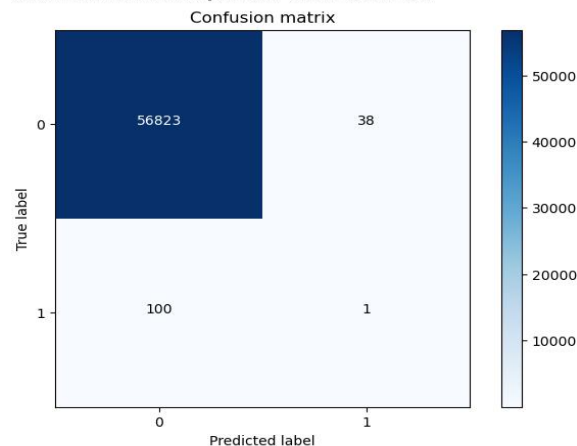
# Compute confusion matrix
cnf_matrix = confusion_matrix(labels_test,y_pred)
np.set_printoptions(precision=2)

print("Recall metric in the testing dataset: ", cnf_matrix[1,1]/(cnf_matrix[1,0]+cnf_matrix[1,1]))

# Plot non-normalized confusion matrix
class_names = [0,1]
plt.figure()
plot_confusion_matrix(cnf_matrix
                      , classes=class_names
                      , title='Confusion matrix')
plt.show()

```

Recall metric in the testing dataset: 0.009900990099009901



8. 简述参数 C 与 gamma 的意义。

#223428010210 陈梓欣

支持向量机 (SVM) 中参数 C 与 gamma 的意义

在支持向量机 (SVM) 中, C 和 gamma 是两个关键超参数, 影响模型的拟合能力与泛化性能。

◆ 参数 C (惩罚系数)

- 作用: 控制对误分类样本的惩罚力度。
- 含义:
 - C 值 越大:
 - 更严格分类
 - 模型更倾向于完全拟合训练集
 - 容易出现 过拟合

▪ C 值 越小:

- 允许部分误分类
- 更强的泛化能力
- 模型可能 欠拟合

类比: C 像“纪律分数”, 越高要求越严格, 越低容忍度越大。

◆ 参数 gamma (核函数参数)

• 作用: 控制高斯核函数 (如 RBF) 中每个训练样本的影响范围。

• 含义:

▪ gamma 值 越大:

- 样本影响范围越小, 模型关注局部
- 决策边界更复杂
- 易 过拟合

▪ gamma 值 越小:

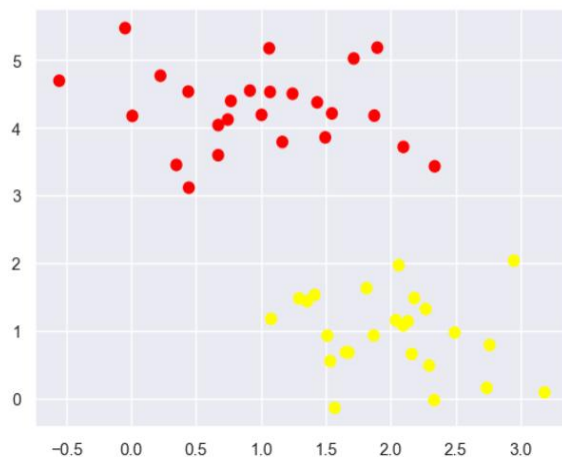
- 样本影响范围更大, 模型更加平滑
- 易 欠拟合

类比: gamma 像“放大镜”, 大 gamma 关注细节, 小 gamma 观察全局趋势。

9. 通过 make_blobs 建立模拟数据并完成下面操作:

(1) 对数据进行可视化显示;

```
# 223428010210 陈梓欣
# 随机点数据, 构造数据集
# 样本点数, 多少堆, 离散程度
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=50, centers=2,
                  random_state=0, cluster_std=0.60)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plt.show()
```



(2) 通过 SVC 线性分类方法对数据集进行分类并可视化显示决策边界;

```
# 223428010210 陈梓欣
# SVC 线性分类
from sklearn.svm import SVC
# 线性向量化
model = SVC(kernel='linear')
model.fit(X, y)
```

SVC

SVC(kernel='linear')

```
# 223428010210 陈梓欣
# 绘图函数
def plot_svc_decision_function(model, ax=None, plot_support=True):
    """Plot the decision function for a 2D SVC"""
    if ax is None:
        ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # create grid to evaluate model
    x = np.linspace(xlim[0], xlim[1], 30)
```

```

y = np.linspace(ymin[0], ymax[1], 30)
Y, X = np.meshgrid(y, x)
xy = np.vstack([X.ravel(), Y.ravel()]).T
P = model.decision_function(xy).reshape(X.shape)

# plot decision boundary and margins
ax.contour(X, Y, P, colors='k',
           levels=[-1, 0, 1], alpha=0.5,
           linestyles=['--', '-', '--'])

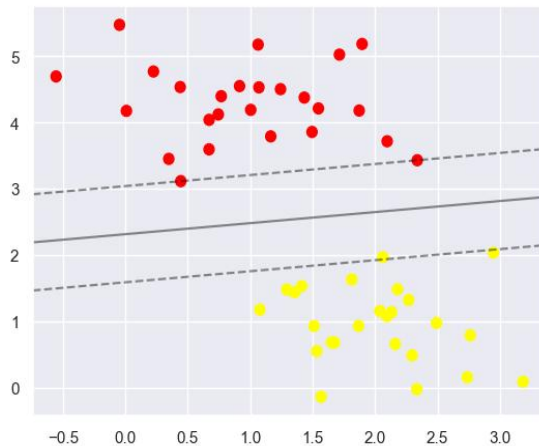
# plot support vectors
if plot_support:
    ax.scatter(model.support_vectors_[:, 0],
               model.support_vectors_[:, 1],
               s=300, linewidth=1, facecolors='none')
ax.set_xlim(xlim)
ax.set_ylim(ylim)
plt.show()

```

```

# 223428010210 陈梓欣
# 绘制散点图
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(model)

```



(3) 修改数据集的散度为 0.8，参数 C 选取 10 和 0.1 对比分类情况；

```

# 223428010210 陈梓欣

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 1. 生成数据集 (散度为0.8)
X, y = make_blobs(n_samples=50, centers=2,
                  random_state=0, cluster_std=0.8)

# 2. 定义绘图函数
def plot_svm(clf, X, y, title):
    plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')

    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # 创建网格以评估模型
    xx = np.linspace(xlim[0], xlim[1], 30)
    yy = np.linspace(ylim[0], ylim[1], 30)
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = clf.decision_function(xy).reshape(XX.shape)

    # 绘制决策边界和边界线
    ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1],
               alpha=0.7, linestyles=['--', '-', '--'])

    # 支持向量点
    ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
               s=100, linewidth=1, facecolors='none', edgecolors='k')

```



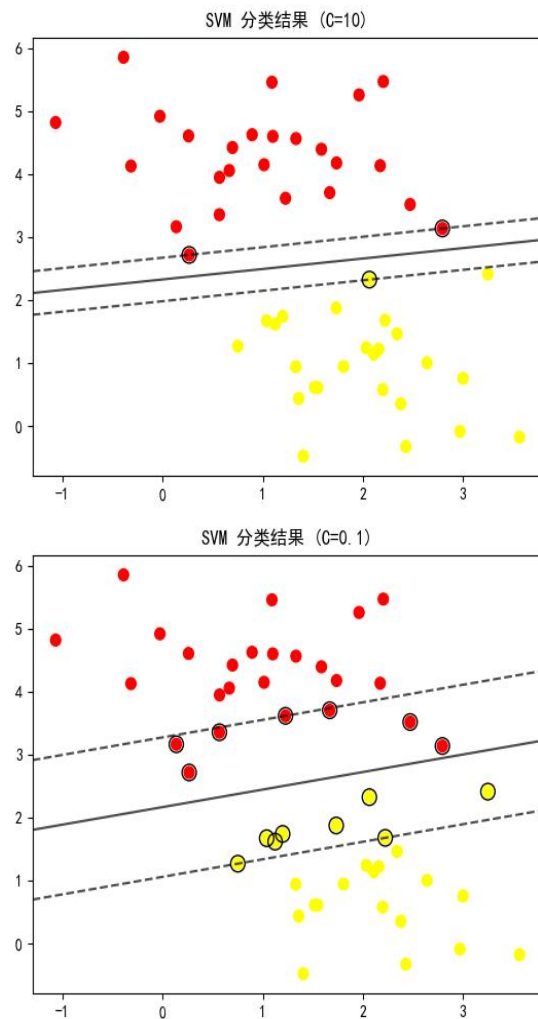
```

plt.title(title)
plt.show()

# 3. C = 10 (高惩罚, 拟合更紧)
clf_c10 = SVC(kernel='linear', C=10)
clf_c10.fit(X, y)
plot_svm(clf_c10, X, y, "SVM 分类结果 (C=10)")

# 4. C = 0.1 (低惩罚, 容错更多)
clf_c01 = SVC(kernel='linear', C=0.1)
clf_c01.fit(X, y)
plot_svm(clf_c01, X, y, "SVM 分类结果 (C=0.1)")

```



10. 通过 `make_circles` 方法建立模拟数据并完成下面操作:

(1) 对数据进行可视化显示;

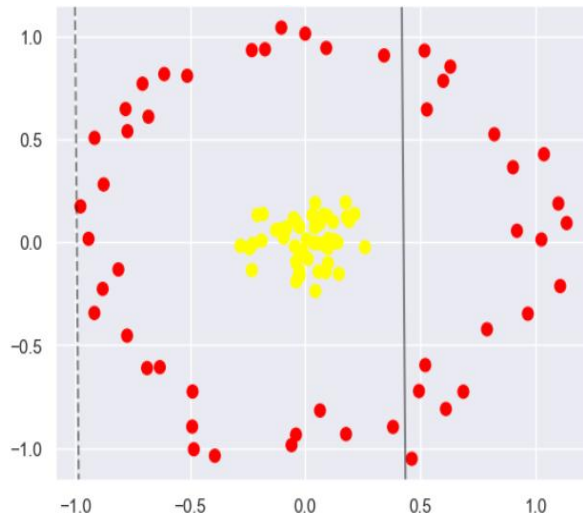
```

# 223428010210 陈梓欣
# 非线性分类SVM
# 引入核函数的SVM
from sklearn.datasets import make_circles
X, y = make_circles(100, factor=.1, noise=.1)

clf = SVC(kernel='linear').fit(X, y)

plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(clf, plot_support=False);

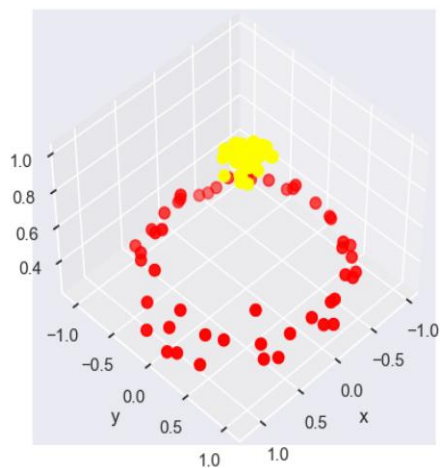
```



(2) 在三维空间中对数据可视化显示;

```
# 223428010210 陈梓欣
# 试试高维核变换
# 加入了新的维度r
from mpl_toolkits import mplot3d
r = np.exp(-(X ** 2).sum(1))
def plot_3D(elev=30, azim=30, X=X, y=y):
    ax = plt.subplot(projection='3d')
    ax.scatter3D(X[:, 0], X[:, 1], r, c=y, s=50, cmap='autumn')
    ax.view_init(elev=elev, azim=azim)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('r')

plot_3D(elev=45, azim=45, X=X, y=y)
plt.show()
```



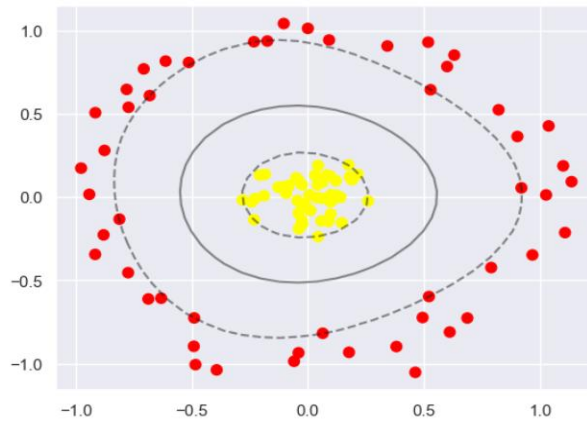
(3) 通过 SVC rbf 分类方法对数据集进行分类并可视化显示决策边界;

```
# 223428010210 陈梓欣
# 加入径向基函数, 高斯核函数, RBF
clf = SVC(kernel='rbf', C=1E6)
clf.fit(X, y)
```

SVC

SVC(C=1000000.0)

```
# 223428010210 陈梓欣
# 绘制分类结果散点图
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(clf)
plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
            s=300, lw=1, facecolors='none');
```



11. 通过 `make_blobs` 建立散度为 1.1 的模拟数据集，应用 `SVC rbf` 分类方法，对比 `gamma` 参数取值为 10 和 0.1 情况下的分类情况并可视化展示。

```
# 223428010210 陈梓欣

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 1. 构造散度为1.1的数据集
X, y = make_blobs(n_samples=100, centers=2,
                  random_state=42, cluster_std=1.1)

# 2. 定义绘图函数
def plot_svm_rbf(clf, X, y, title):
    plt.figure(figsize=(6,4))
    plt.scatter(X[:, 0], X[:, 1], c=y, s=40, cmap='autumn')

    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # 创建网格用于绘制决策边界
    xx = np.linspace(xlim[0], xlim[1], 500)
    yy = np.linspace(ylim[0], ylim[1], 500)
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = clf.decision_function(xy).reshape(XX.shape)

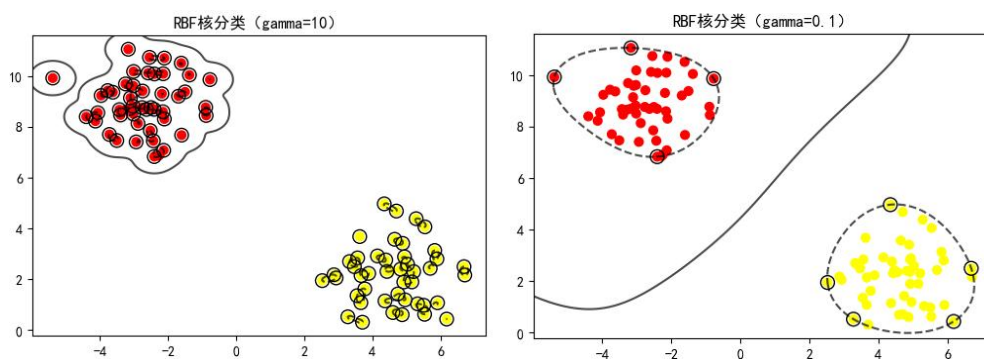
    # 绘制等高线
    ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1],
              alpha=0.7, linestyles=['--', '-', '--'])

    # 绘制支持向量
    ax.scatter(clf.support_vectors[:, 0], clf.support_vectors[:, 1],
              s=100, facecolors='none', edgecolors='k', linewidths=1)

    plt.title(title)
    plt.show()

# 3. gamma = 10 (高复杂度)
clf_gamma10 = SVC(kernel='rbf', gamma=10)
clf_gamma10.fit(X, y)
plot_svm_rbf(clf_gamma10, X, y, "RBF核分类 (gamma=10)")

# 4. gamma = 0.1 (边界更平滑)
clf_gamma01 = SVC(kernel='rbf', gamma=0.1)
clf_gamma01.fit(X, y)
plot_svm_rbf(clf_gamma01, X, y, "RBF核分类 (gamma=0.1)")
```



五、实验中遇到的问题及解决方法

问题：matplotlib 中文乱码或图像显示异常。

解决方法：代码前加入两行设置即可，代码如下：

```
plt.rcParams['font.sans-serif'] = ['SimHei']  
plt.rcParams['axes.unicode_minus'] = False。
```

问题：数据不平衡导致模型偏向多数类。

解决方法：使用欠采样、过采样或设置 `class_weight='balanced'` 解决类别不均衡问题。

问题：SVM 训练时间过长或分类效果差。

解决方法：确保特征已标准化，并调节 `C` 和 `gamma`。高 `gamma` 会导致过拟合，低 `gamma` 容易欠拟合。

六、实验总结及心得

通过本次“信用卡欺诈检测与 SVM 参数应用”实验，我深入理解了 Logistic 回归与支持向量机（SVM）模型的基本原理、超参数含义及实际应用过程。在处理 `creditcard.csv` 数据过程中，体会到了真实金融数据中类别极度不平衡的问题，并通过欠采样、过采样等方式进行平衡处理。通过标准化、删除无关列等数据预处理操作，提升了建模效果的准确性与稳定性。

在 Logistic 回归模型中，我通过对不同 `C` 值的设置，观察模型的分类性能变化，理解了 `C` 作为正则化参数在调控过拟合和欠拟合中的作用。结合混淆矩阵与热力图展示，使结果分析更加直观。随后，通过 SVC 进行线性与非线性分类实验，尤其在对 `make_blobs` 和 `make_circles` 模拟数据进行核函数分类测试中，进一步掌握了 SVM 中 `C` 与 `gamma` 两个关键参数的调节影响及 RBF 核的工作原理。

整体而言，本次实验结合真实数据与可控模拟数据，使我在实践中深化了对分类算法与模型调参的理解，也提升了我对金融风控建模的基本能力和数据科学应用素养。